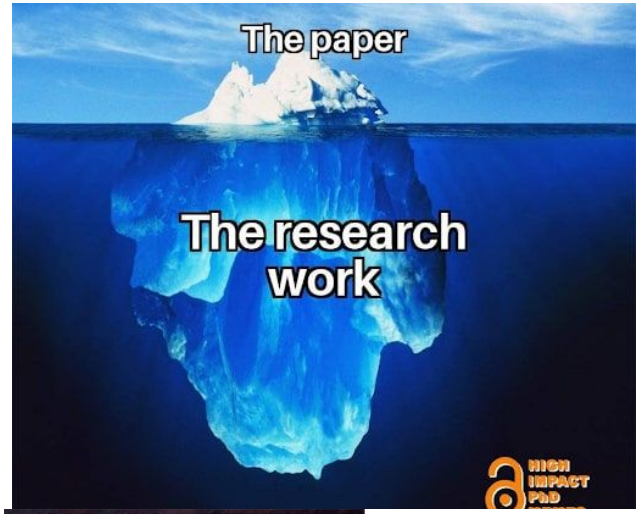
Multimodal Dataset Quantization

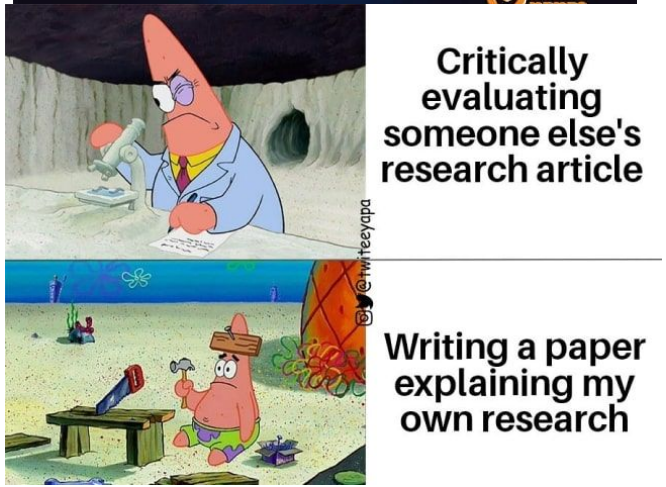
Viktor Moskvoretskii
Supervisor: Narek Alvandian

Ремарка для проверяющих

- Данная работа больше академическая
- Авторы испытывают страсть к науке и хотят сделать что-то новое и полезное
- Оно будет переложено на практику и имеет практическое применение
- Но затрудняется работа тем, что мы первые кто делает подобное

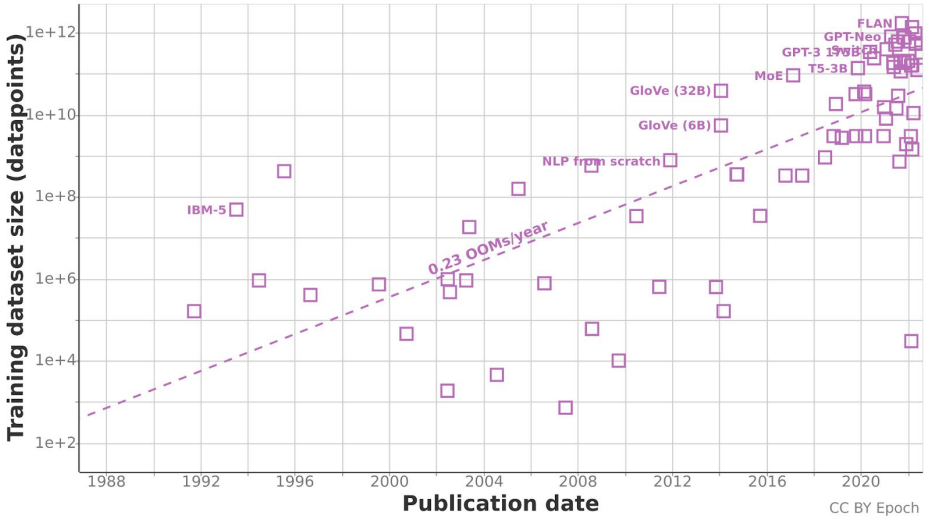
Git: https://github.com/VityaVitalich/MultiModalCoreset





В чем проблема?

- В мире много данных
- Они тяжелые
- Особенно мультимодальные
- Долго тренировать модели



[1] ImageNet-21K Pretraining for the Masses, 2021
[2] 4M: Massively Multimodal Masked Modeling, 2023
[3] On the Sentence Embeddings from Pre-trained Language Models, 2020
[4]Simple and Effective Prevention of Mode Collapse in Deep One-Class Classification
[5] LAION-5B: An open large-scale dataset for training next generation image-text models, 2022

Что предлагаем?

- Опереться на статью [2]
- Набрать объектов так, чтобы
  - максимизировать разнообразие
  - минимизировать расстояние до всей выборки
- Таким образом наберем сабсет такой же хороший как вся выборка
- Меньше хранить
- Меньше учить
- А главное - сделать мультимодальным

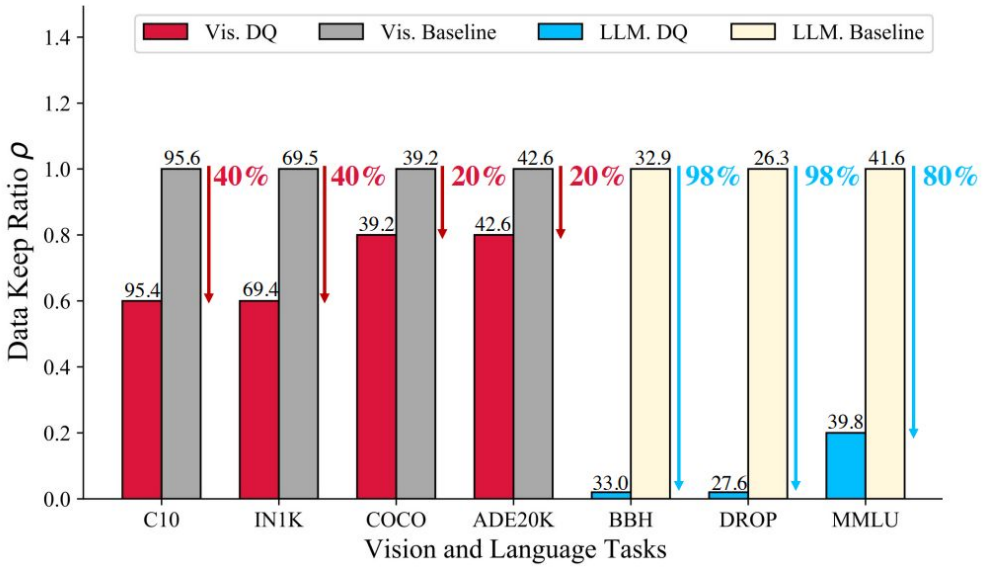


График из статьи, проценты - на сколько сократилась выборка.

Цифры над столбцами - метрики на тесте

[1] Submodular Combinatorial Information Measures with Applications in Machine Learning, 2021
[2] Dataset Quantization, 2023

Что предлагаем?

- Взять простой мультимодал датасет
  - RGB изображение
  - Semantic маска
  - Глубина
- Взять хорошую бейзлайн мультимодальную модель
  - MultiMAE [1]
- Набрать разными способами сабсеты и проверить как они работают
  - Если работают порадоваться и расширить на больше данных
  - Если не работают попытаться еще
  - Если совсем не работают провести анализ и обосновать
- Подумать и попробовать как Lossy Compression мог бы нам помочь в нашей задаче

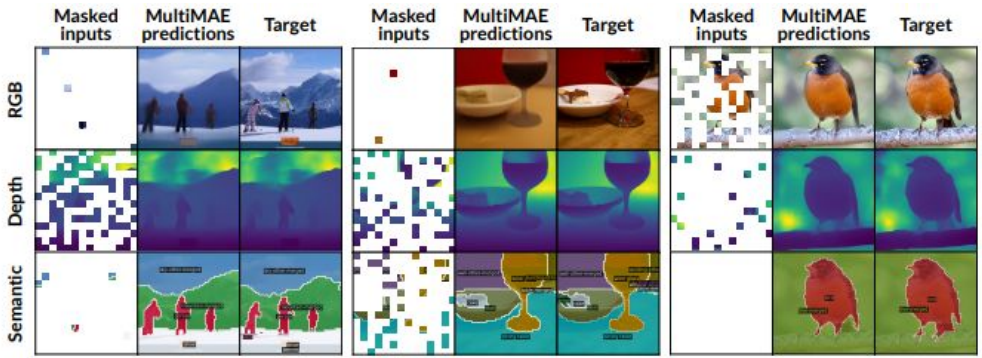


Figure 1. **MultiMAE pre-training objective.** We randomly select 1/6 of all 16×16 image patches from multiple modalities and learn to reconstruct the remaining 5/6 masked patches from them. The figure shows validation examples from ImageNet, where masked inputs (left), predictions (middle), and non-masked images (right) for RGB (top), depth (middle), and semantic segmentation (bottom) are provided. Since we do not compute a loss on non-masked patches, we overlay the input patches on the predictions.

[1] Bachmann, Roman, et al. "Multimae: Multi-modal multi-task masked autoencoders." *European Conference on Computer Vision*. Cham: Springer Nature Switzerland, 2022.

Первые эксперименты с сабсетами

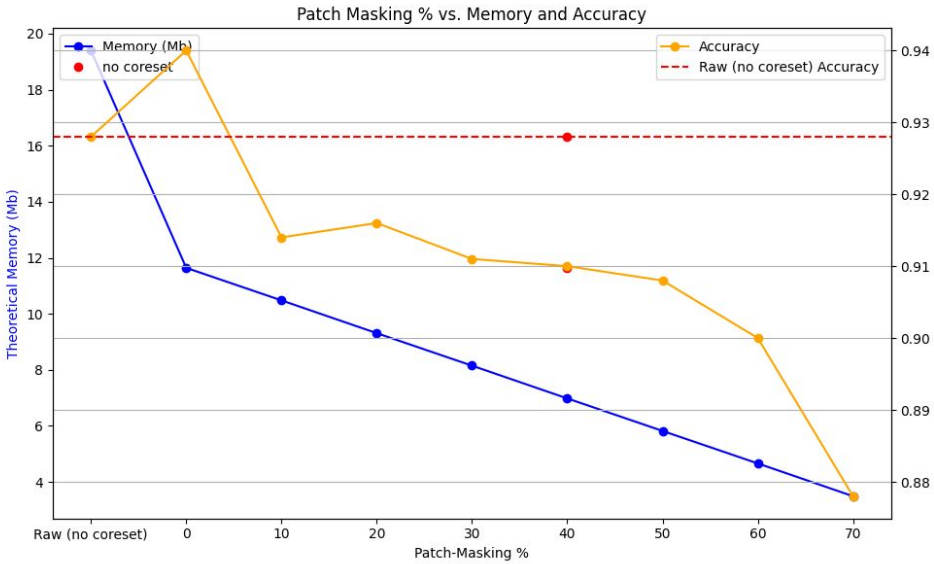
Работа с унимодальными сабсетами

- Воспроизвели статью про сабсеты на Cifar-10
- Результаты сошлись со статьей
- На Imagenet-1 считается очень долго, пока не воспроизвели

```python
from validate_cifar import run_experiment
import os

ratios_acc = []
if __name__ == "__main__":
    ratios = [0.1, 0.2, 0.3, 0.4, 0.5, 0.6, 0.7]

    for mask_ratio in ratios:
        saved_path = f"./results/pixel_quantized_cifar_{mask_ratio}"
        if not os.path.exists(saved_path):
            os.makedirs(saved_path, exist_ok=True)

            os.system(
                f"python -u quantize_pixel.py     --data CIFAR10 --data_path /home/data/dq/cifar-10-batches-py/  \
            --output_dir ./results/pixel_quantized_cifar_{mask_ratio} --model mae_vit_large_patch16   \
                --resume ./pretrained/mae_visualize_vit_large_ganloss.pth --batch_size 512      --mask_ratio {mask_ratio} --cam_mask"
            )

        select_indices_path = (
            "./results/sample_quantized_cifar_010/select_indices_CIFAR10_0.6.npy"
        )
        best_acc = run_experiment(
            lr=0.1,
            bs=128,
            data_dir=saved_path,
            selected_indices=[select_indices_path],
            result_path="",
        )
        ratios_acc.append(best_acc)

    print(ratios_acc)
```

Как влияет маскирование патчей

- Мы задумались что лучше, резать семплы или маскировать часть патчей?
- Для ответа решили посмотреть на экономию по памяти на диске и итоговое качество на тест сете

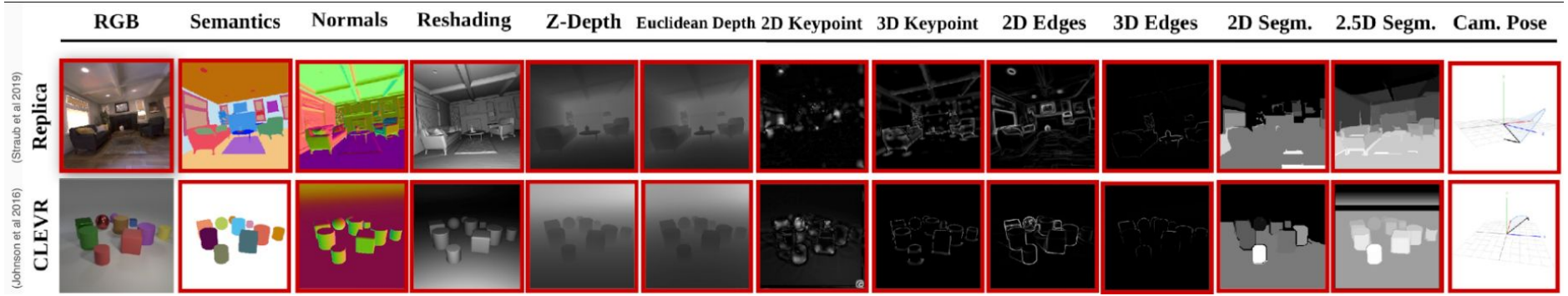


- Результаты показывают, что при наборе сабсета без маскирования, мы экономим 8 Мб, при этом качество вырастает на 1% от базового
- При маскировании без сабсета для тех же результатов по памяти нам нужно замаскировать 40%, при этом качество совпадает с изначальным
- При наборе сабсета и маскировании качество начинает падать от 1.5% и больше
- Однако экономия по памяти не такая большая
- Трейд-офф зависит от того что нам важнее

Эксперименты с MultiMAE

Датасет

- Пока что только CLEVR
- 70K изображений 512 x 512
- Разные фигурки на одинаковом фоне
- Есть много модальностей
- Одинаковое распределение пикселей
- Хорошая модельная задача чтобы все протестировать



Метрики и постановки

Metrics

- RMSE = Root Mean Squared Error
- Delta-1 = Percentage of pixels with error $max(\frac{\hat{y}}{y}, \frac{y}{\hat{y}})$ more than 1.25

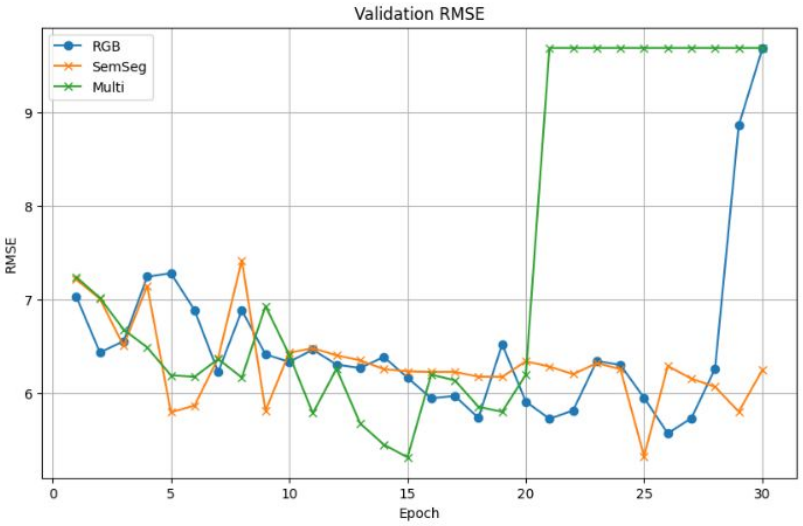
Parameters:

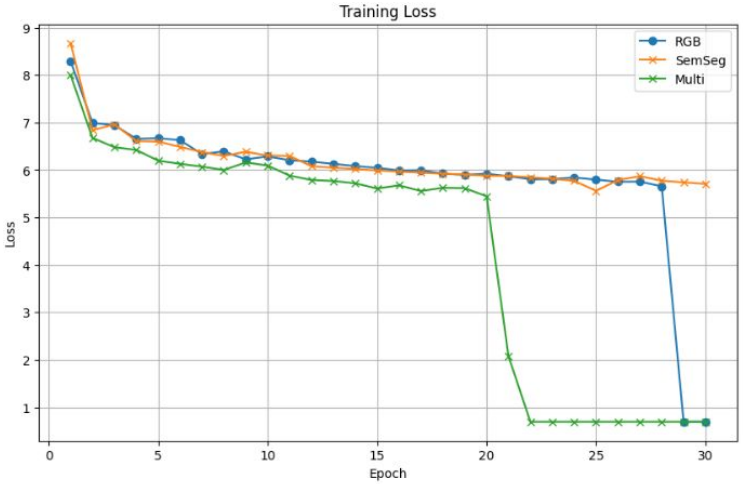
- Batch size = 256
- 30 Epochs
- lr = 3e-4
- decay = 1e-4
- Images rescaled to 224 x 224
- Patch size 16

Input domain	Delta-1	RMSE
RGB	1.0	263
RGB + Semantic	1.0	204
Semantic	1.0	206

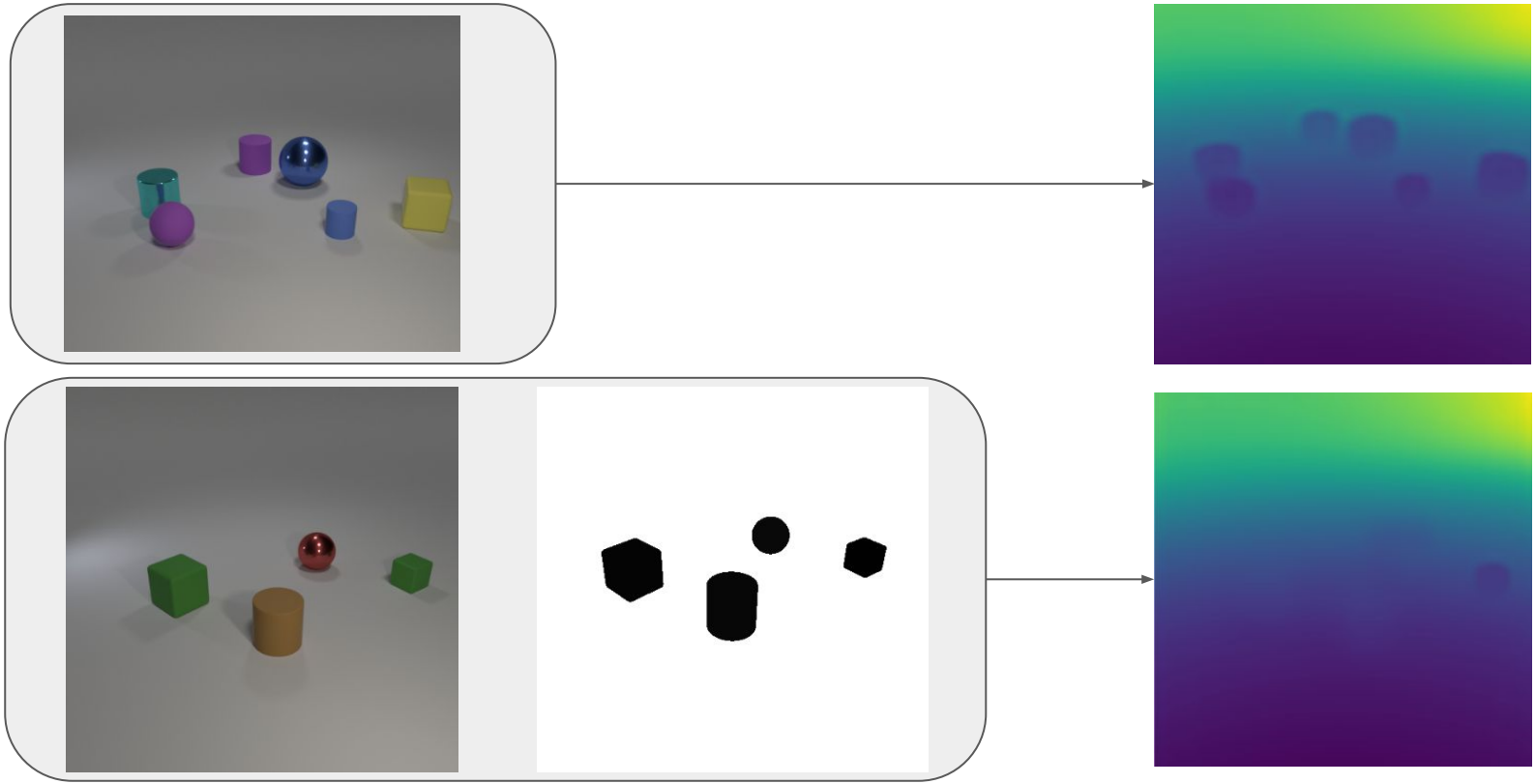
Динамика обучения

- Видно что переобучаемся в один момент
- Y-шкала логарифмирована для удобства
- странная динамика на валидации
- Необычно что метрики хорошие а примеры живые не очень (будем фиксить)
- В целом ожидаемо по метрикам мультимодал лучше унимодала





Примеры



Устройство кода

MultiMAE

Основные пункты

- Описаны в README в репо
- Продублированы сюда для удобства
- Код разделен на части
  - MultiMAE
  - Dataset Quantization
  - Bot

Как учить MultiMAE

- Поменять конфиг на пути до своих данных
- Выставить желаемые параметры обучения
- Запустить run_finetuning_depth.py
- Смотреть логи в папке logs/
- Искать чекпоинт в папке ckpt/

```python
def depth_configs():
    config = ml_collections.ConfigDict()

    config.run_name = "debug-only-semseg"
    config.log_dir = "./logs/"
    config.cons_lvl = "INFO"
    config.file_lvl = "INFO"

    config.device = "cuda"

    config.in_domains = ["semseg"]
    config.out_domains = ["depth"]
    config.decoder_main_tasks = ["semseg"]

    config.semseg_num_classes = 256

    config.patch_size = 16
    config.input_size = 224

    config.fine_tune_path = "../../data/dq/mae-b_dec512d8b_1600e_multivit-c477195b.pth"

    config.lr = 3e-4
    config.weight_decay = 1e-4
    config.total_epochs = 5
    config.batch_size = 64

    config.train_dir = "../../data/dq/clevr_complex/train"
    config.val_dir = "../../data/dq/clevr_complex/val"

    return config
```

Пример лога

```
- 2023-12-24 02:07:57,009 - Epoch: 2; metrics on val: {'rmse': 492.3342603938595, 'rel': 0.023249789586017845, 'srel': 15.239688375193602, 'log10': 0.03209162734828557, 'delta_1': 0.9992959746129954, '
- 2023-12-24 02:07:58,657 - Epoch 0003: train started
- 2023-12-24 04:07:03,098 - Epoch 0003: avg train loss = 433.3
- 2023-12-24 04:07:03,098 - Epoch 0003: train finished
- 2023-12-24 04:07:03,099 - Epoch 0003: validation started
- 2023-12-24 04:17:24,707 - Epoch 0003: validation metrics: {'rmse': 492.2877232253931, 'rel': 0.02810924490735789, 'srel': 15.065610229589378, 'log10': 0.03244304108866461, 'delta_1': 0.99952012756068
- 2023-12-24 04:17:24,707 - Epoch 0003: validation finished
- 2023-12-24 04:17:24,708 - Epoch: 3; metrics on val: {'rmse': 492.2877232253931, 'rel': 0.02810924490735789, 'srel': 15.065610229589378, 'log10': 0.03244304108866461, 'delta_1': 0.9995201275606823, 'de
- 2023-12-24 04:17:26,294 - Epoch 0004: train started
- 2023-12-24 06:17:00,499 - Epoch 0004: avg train loss = 344.4
- 2023-12-24 06:17:00,499 - Epoch 0004: train finished
- 2023-12-24 06:17:00,500 - Epoch 0004: validation started
- 2023-12-24 06:27:32,638 - Epoch 0004: validation metrics: {'rmse': 366.5742035884007, 'rel': 0.023092109057459103, 'srel': 8.534470828475467, 'log10': 0.025940898105881776, 'delta_1': 0.9999300313603
- 2023-12-24 06:27:32,638 - Epoch 0004: validation finished
- 2023-12-24 06:27:32,639 - Epoch: 4; metrics on val: {'rmse': 366.5742035884007, 'rel': 0.023092109057459103, 'srel': 8.534470828475467, 'log10': 0.025940898105881776, 'delta_1': 0.9999300313603346, '
- 2023-12-24 06:27:34,231 - Epoch 0005: train started
- 2023-12-24 08:29:03,588 - Epoch 0005: avg train loss = 281.8
- 2023-12-24 08:29:03,588 - Epoch 0005: train finished
- 2023-12-24 08:29:03,589 - Epoch 0005: validation started
- 2023-12-24 08:39:37,110 - Epoch 0005: validation metrics: {'rmse': 371.71744474180184, 'rel': 0.01978406304149491, 'srel': 7.534697505319194, 'log10': 0.0222652018022765, 'delta_1': 0.999976415163392
- 2023-12-24 08:39:37,110 - Epoch 0005: validation finished
- 2023-12-24 08:39:37,178 - Epoch: 5; metrics on val: {'rmse': 371.71744474180184, 'rel': 0.01978406304149491, 'srel': 7.534697505319194, 'log10': 0.0222652018022765, 'delta_1': 0.9999764151633925, 'de
- 2023-12-24 08:39:38,755 - run 'long-rgb-semseg_2023-12-23_21:46:00' finished successfully
```

Как раскатить на другие лоссы, модальности и тд

- Внимательно изучить базовый трейнер
- Увидеть что там не дописаны функции лосса и логгирования
- Отнаследоваться от базового трейнера
  - Как это сделано в RgbDepthTrainer
- Написать желаемый лосс и возвращаемые метрики
- Если очень хочется можно исправить форвард модели в трейне

```python
class BaseTrainer:
    """A base class for all trainers."""

    def __init__(
        self,
        *,
        model: nn.Module,
        optimizer: Union[torch.optim.Optimizer, None] = None,
        lr_scheduler: Union[torch.optim.lr_scheduler._LRScheduler, None] = None,
        train_loader: Union[DataLoader, None] = None,
        val_loader: Union[DataLoader, None] = None,
        run_name: Union[str, None] = None,
        total_iters: Union[int, None] = None,
        total_epochs: Union[int, None] = None,
        iters_per_epoch: Union[int, None] = None,
        ckpt_dir: Union[str, os.PathLike, None] = None,
        ckpt_replace: bool = False,
        ckpt_track_metric: str = "epoch",
        ckpt_resume: Union[str, os.PathLike, None] = None,
        device: str = "cpu",
        metrics_on_train: bool = False,
    ):
        """Initialize trainer.

        Args:
            model: model to train or validate.
            optimizer: torch optimizer for training.
            lr_scheduler: torch learning rate scheduler.
            train_loader: train dataloader.
            val_loader: val dataloader.
            run_name: for runs differentiation.
            total_iters: total number of iterations to train a model.
            total_epochs: total number of epoch to train a model. Exactly one of
                `total_iters` and `total_epochs` shoud be passed.
            iters_per_epoch: validation and checkpointing are performed every
                `iters_per_epoch` iterations.
            ckpt_dir: path to the directory, where checkpoints are saved.
            ckpt_replace: if `replace` is `True`, only the last and the best checkpoint
                are kept in `ckpt_dir`.
            ckpt_track_metric: if `ckpt_replace` is `True`, the best checkpoint is
                determined based on `track_metric`. All metrcs except loss are assumed
                to be better if the value is higher.
            ckpt_resume: path to the checkpoint to resume training from.
            device: device to train and validate on.
            metrics_on_train: wether to compute metrics on train set.
            model_conf: Model configs from configs/ dir
            data_conf: Data configs from configs/ dir
        """
```

Как раскатить на другие лоссы, модальности и тд

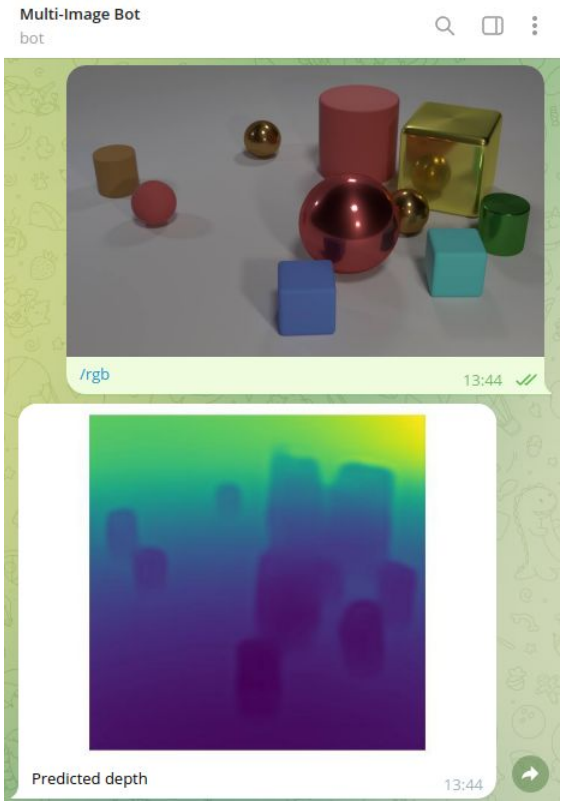
- Если нужны новые модальности посмотреть dataset.py
- Добавить туда обработку новых модальностей, если их еще нет. Есть:
  - RGB
  - Semantic Mask
  - Depth
- Если надо добавить Transforms в run_finetuning_depth.py

```python
class MultiModalDataset(Dataset):
    def __init__(
        self,
        root_dir,
        train_transform=None,
        target_transofrm=None,
        input_tasks=["rgb"],
        output_task="depth",
    ):
        self.root_dir = root_dir
        self.train_transform = train_transform
        self.target_transform = target_transofrm
        self.input_tasks = input_tasks
        self.output_task = output_task
        self.all_tasks = input_tasks + [output_task]
        for task_name in self.all_tasks:
            task_path = os.path.join(root_dir, task_name)
            self.__dict__[task_name + "_path"] = task_path
            self.__dict__[task_name + "_files"] = sorted(os.listdir(task_path))

    def __len__(self):
        task = self.all_tasks[0]
        return len(self.__dict__[task + "_files"])

    def __getitem__(self, idx):
        # obtain input object
        sample_dict = {}
        for task in self.input_tasks:
            file_name = self.__dict__[task + "_files"][idx]
            file_path = os.path.join(self.__dict__[task + "_path"], file_name)
            x = Image.open(file_path)
            if task == "rgb":
                x = x.convert("RGB")
            if task == "semantic":
                x = x.convert("L")

            if self.train_transform:
                x = self.train_transform[task](x)
            sample_dict[task] = x
```

Tg Bot

Как запустить

- В директории bot/ объявить переменной свой токен
- запустить bot.py
- искать нужные ручки в хендлерах
- менять обработку модели в model_utils.py



Dataset Quantization

Как запускать

- Подробная инструкция в README
- Если кратко
  - Скачать цифар и положить в data_cifar (подробно написано в репо)
  - Запустить скрипт набора бинов с данными
  - Запустить скрипт семплинга из набранных бинов
  - Запустить скрипт маскирования с помощью MAE
  - Запустить скрипт обучающий ResNet и считающий метрики

```
# Dataset bin generation (By default we use a bin number of 10)
CUDA_VISIBLE_DEVICES=0 python -u quantize_sample.py \
    --fraction 0.1 --dataset CIFAR10 --data_path ~/data_cifar \
    --num_exp 10 --workers 10 -se 0 --selection Submodular --model ResNet18 \
    -sp ../results/bin_cifar_010 \
    --batch 128 --submodular GraphCut --submodular_greedy NaiveGreedy --pretrained

# Bin sampling (Change the fraction parameter to obtain different data keep ratio)
CUDA_VISIBLE_DEVICES=0 python -u quantize_bin.py \
    --fraction 0.125 --dataset CIFAR10 --data_path ~/data_cifar \
    --workers 10 --selection_path ../results/bin_cifar_010/ \
    -sp ../results/sample_quantized_cifar_010

# Image reconstruction (Change the mask_ratio parameter to obtain different patch drop ratio)
CUDA_VISIBLE_DEVICES=0 python -u quantize_pixel.py \
    --data CIFAR10 --data_path ~/data_cifar \
    --output_dir ../results/pixel_quantized_cifar --model mae_vit_large_patch16 \
    --resume ./pretrained/mae_visualize_vit_large_ganloss.pth --batch_size 128 \
    --mask_ratio 0.2 --cam_mask
```

Следующие шаги

- Понять в чем беда с миксом модальностей
  - пытаемся починить давно
  - параметры обучения меняли
  - багов не нашли
- Если не починится, попробовать
  - другой датасет
  - другой набор модальностей
- Протестировать придуманные функции сбора мультимодального сабсета
  - у нас они есть, и их несколько
  - Не знаем какая лучше
  - Одна из важных деталей исследования
- Раскатить на больше модальностей, очень хотим текст
- Выдавить качество, сделать удобно и полезно, попасть на ICLR2025
- Опционально: Попробовать внедрить Lossy Compression

Thank you for your attention!